

ROSKILDE UNIVERSITY

Complex IT Systems – Practice

CIT/P Portfolio Project

Subproject 1

Database design:

Movie Data Model and Framework Model

Group 16

Frank Tongwa Asongna Georgii Yakovlyev
Tommaso Cosimo Miraglia Anders Zobbe Mortensen

October 6, 2026

Contents

1	Overview	1
2	Application design	1
3	Movie Data Model	1
3.1	Problems in the source schema	1
3.2	Design decisions	2
3.3	ER diagram	2
4	Framework Model	4
4.1	Design decisions	4
4.2	Connection to the Movie Data Model	4
4.3	ER diagram	4
5	Reverse engineered diagram of the full database	6
6	Functionality	7
6.1	Framework functions	7
6.2	Title search	7
6.3	Rating	7
6.4	Names, co-players and name ratings	7
6.5	Similar movies	8
6.6	Text retrieval over the inverted index	8
7	Indexing	8
8	Testing	9
9	Evaluation	9

1 Overview

The database consists of two models integrated in one PostgreSQL database: a *Movie Data Model*, derived from the IMDb and OMDb source data, and a *Framework Model* for users, history, ratings and bookmarks. The framework refers to the movie model only through foreign keys, so the movie model can be rebuilt independently of it. Sections 2–5 document the design, Section 6 the functions that expose the database, Section 7 the indexing and Section 8 the testing. The SQL scripts, the test script and the test output are delivered as separate files.

2 Application design

The application is a movie browser in which a registered user can search for titles (by title and plot) and people (by name), view title pages (poster, plot, genres, rating, alternative titles, cast and characters) and person pages (professions, known-for titles, derived rating), rate titles from 1 to 10, bookmark titles and people, revisit their search history, and state genre, language and region preferences.

This has the following implications for the data model:

- **Plot and poster** are not in the IMDb data, so they are taken from OMDb and stored with the title.
- **nconst and tconst** are kept as keys, as the frontend needs them to link to person and title pages.
- **History** must be linked to the titles and people found, not only to the search string, so past results can be reopened.
- **A ranking of people** requires a person rating, derived from title ratings and stored.
- **Genres, professions, languages and regions** are separate entities rather than free text, which also supports user preferences.

3 Movie Data Model

3.1 Problems in the source schema

The seven source tables mirror IMDb's files and are not in good relational shape:

- comma-separated lists in single columns (**genres**, **primaryprofession**, **knownfortitles**, **directors**, **writers**, **types**, **characters**), violating first normal form;
- text domains repeated in every row (title type, region, language, role category), which is redundant and error-prone;
- overlapping information: **directors** and **writers** occur in both **title_crew** and **title_principals**, and **originalTitle** duplicates the original entry among the alternative titles;
- no usable keys for **title_akas** and **title_principals**;
- dangling references (378 rows of **title_principals** refer to non-existent people).

3.2 Design decisions

Each decision below removes a specific problem of the source schema.

Multi-valued columns become tables. Genres, professions, known-for titles, AKA types and characters were stored as comma-separated lists. Each is moved to its own table with one row per value (`genres_title_basic`, `name_primaryprofession`, `knowfor`, `lang_reg_akas`, `title_character`). This gives first normal form and lets foreign keys and joins work on single values.

Repeated text becomes lookup tables. Title type, region, language, role category, profession, genre and AKA type were repeated as text in every row. They are stored once in lookup tables (`title_basic_type`, `region`, `language`, `category`, `primaryprofession`, `genres`, `title_akas_type`) and referenced by key, so each value has one canonical spelling.

The original title is an AKA. `originaltitle` duplicated the original entry among the alternative titles, so it is removed from `title_basic` and stored as an AKA of type `original`. Every title must therefore have at least one AKA; titles lacking one get it during data cleaning.

Language, region and type of an AKA are a separate relation. An AKA can have several types, so these three values cannot be single foreign keys in `title_akas`. The table `lang_reg_akas` links an AKA to a language, a region and a type, with one row per type.

Principals and crew are merged. Directors and writers occurred in both `title_crew` and `title_principals`. They are stored once in `title_principals`, which gets a surrogate key `principalid` and a flag `is_crew` marking directors and writers that come from `title_crew`.

Plot and poster are title attributes. They come from OMDb and depend only on the title, so they are stored in `title_basic`.

Person ratings are stored. The table `name_ratings` holds, for each person, the vote-weighted average $R_p = \sum_t r_t v_t / \sum_t v_t$ of the ratings of their titles, and the total votes $\sum_t v_t$ as `agg_numvotes`. It is stored rather than computed on demand to make rankings of people cheap.

3.3 ER diagram

Titles are the central entity (Figure 1). A title has one or more AKAs, zero or one rating, any number of genres and one type; episodes refer to their parent series with season and episode number. People are related to titles through `title_principals` (category, job, order, characters) and `knowfor`, and have zero or more professions.

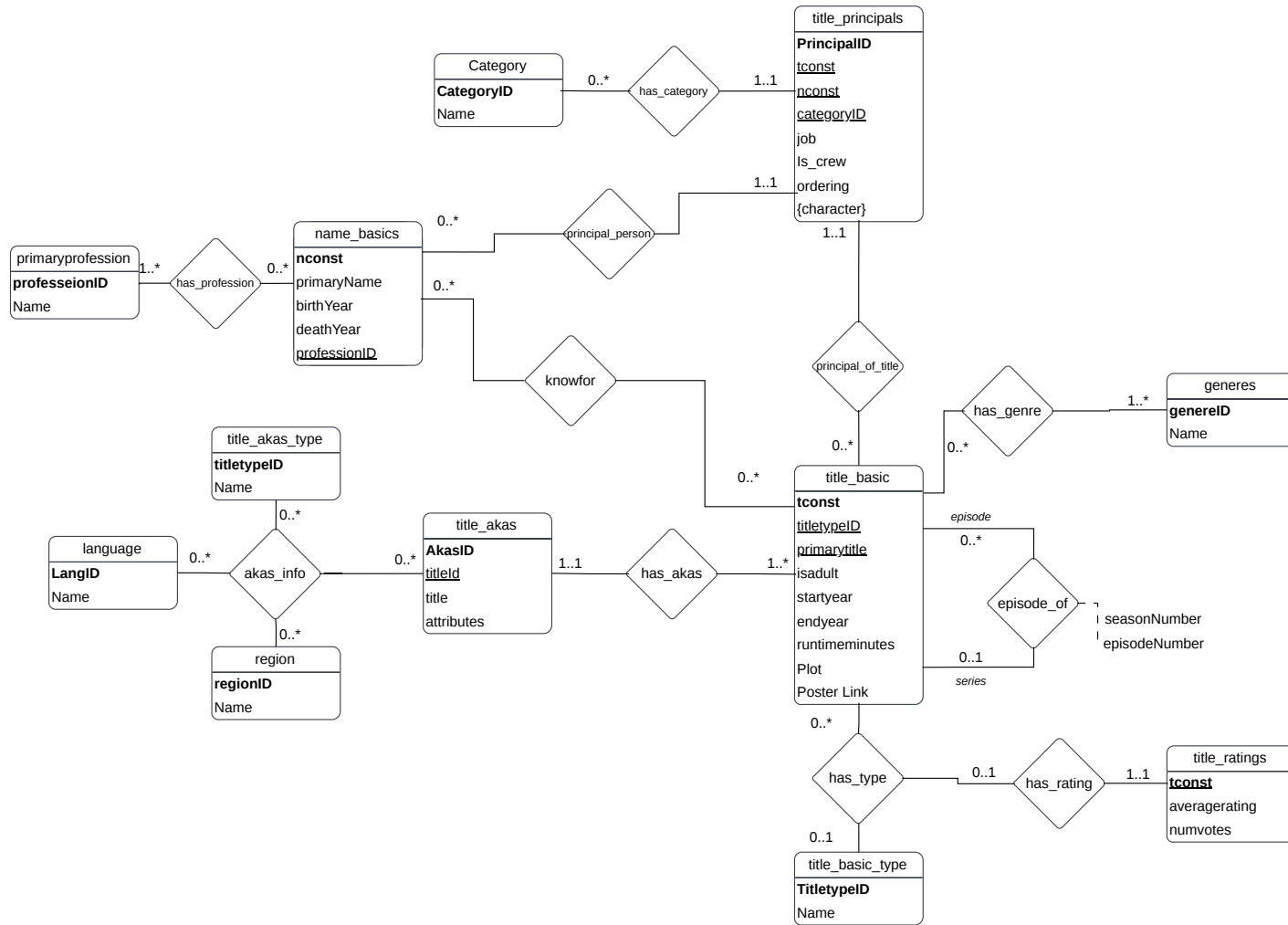


Figure 1: ER diagram of the Movie Data Model.

4 Framework Model

4.1 Design decisions

- **Users.** `app_user` has unique user name and e-mail; optional references to `language` and `region` store preferences. Passwords must be stored as salted hashes by the registering layer.
- **Search history.** One `history` row per search (user, time, search text). The tables `history_title_basic` and `history_name_basic` can link a history entry to the titles and people it returned; these rows are deleted together with their history entry.
- **Ratings.** `rating_history` has key (`userid`, `tconst`), so a user has at most one rating per title, restricted to 1–10 by a `CHECK`; an optional comment is included. This makes repeated rating by the same user consistent by construction.
- **Bookmarks.** `bookmarking_title` and `bookmarking_person` are many-to-many relationships between users and titles/people.
- **Genre preferences.** `genres_user` relates users to genres (own addition).

4.2 Connection to the Movie Data Model

Table 1 shows the only points where the framework touches the movie model.

Framework table	Movie entity	Meaning
<code>app_user</code>	<code>language</code> , <code>region</code>	preferred language and region
<code>history_title_basic</code>	<code>title_basic</code>	titles returned by a search
<code>history_name_basic</code>	<code>name_basics</code>	people returned by a search
<code>rating_history</code>	<code>title_basic</code>	user's rating of a title
<code>bookmarking_title</code>	<code>title_basic</code>	bookmarked title
<code>bookmarking_person</code>	<code>name_basics</code>	bookmarked person
<code>genres_user</code>	<code>genres</code>	preferred genre

Table 1: Connections from the Framework Model to the Movie Data Model.

4.3 ER diagram

Figure 2 shows the Framework Model with the movie entities it connects to.

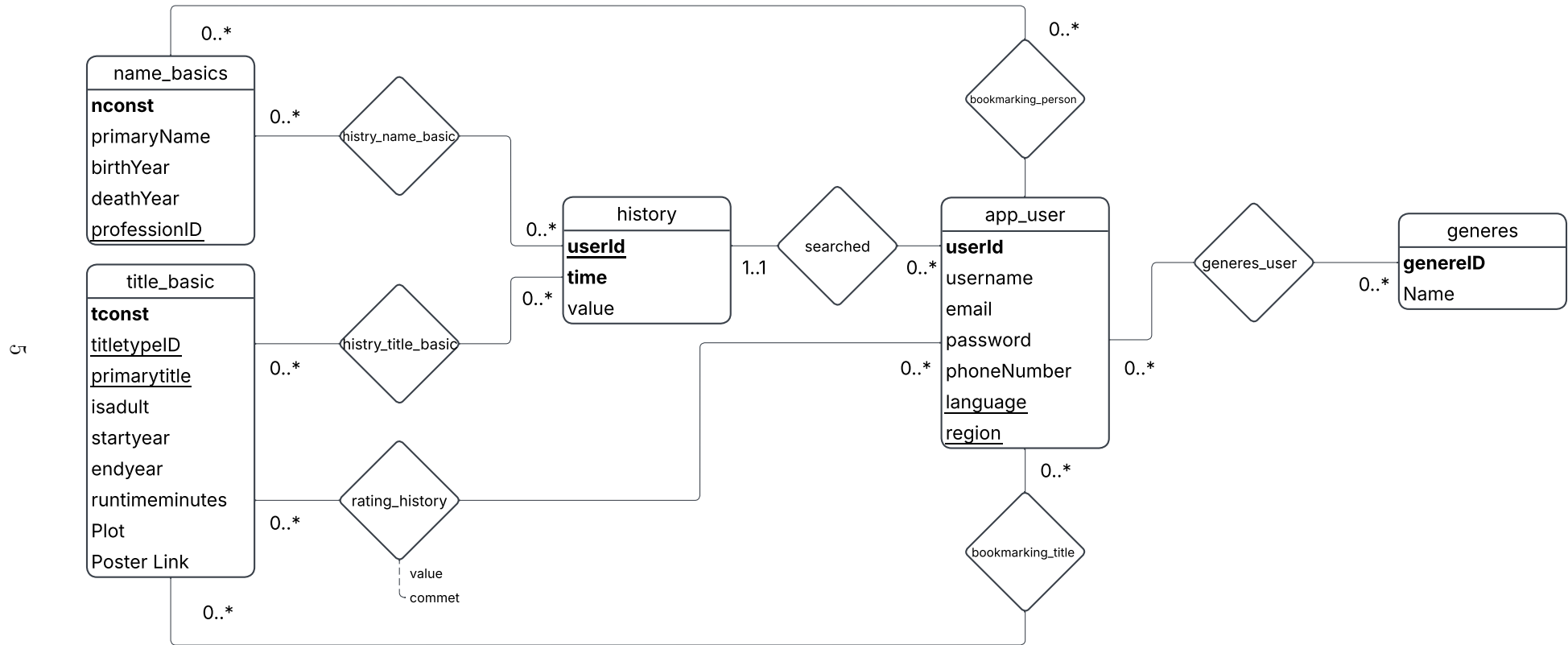


Figure 2: ER diagram of the Framework Model.

5 Reverse engineered diagram of the full database

The complete entity-relationship diagram of the database, integrating both the Movie Data Model and the Framework Model, is presented below.

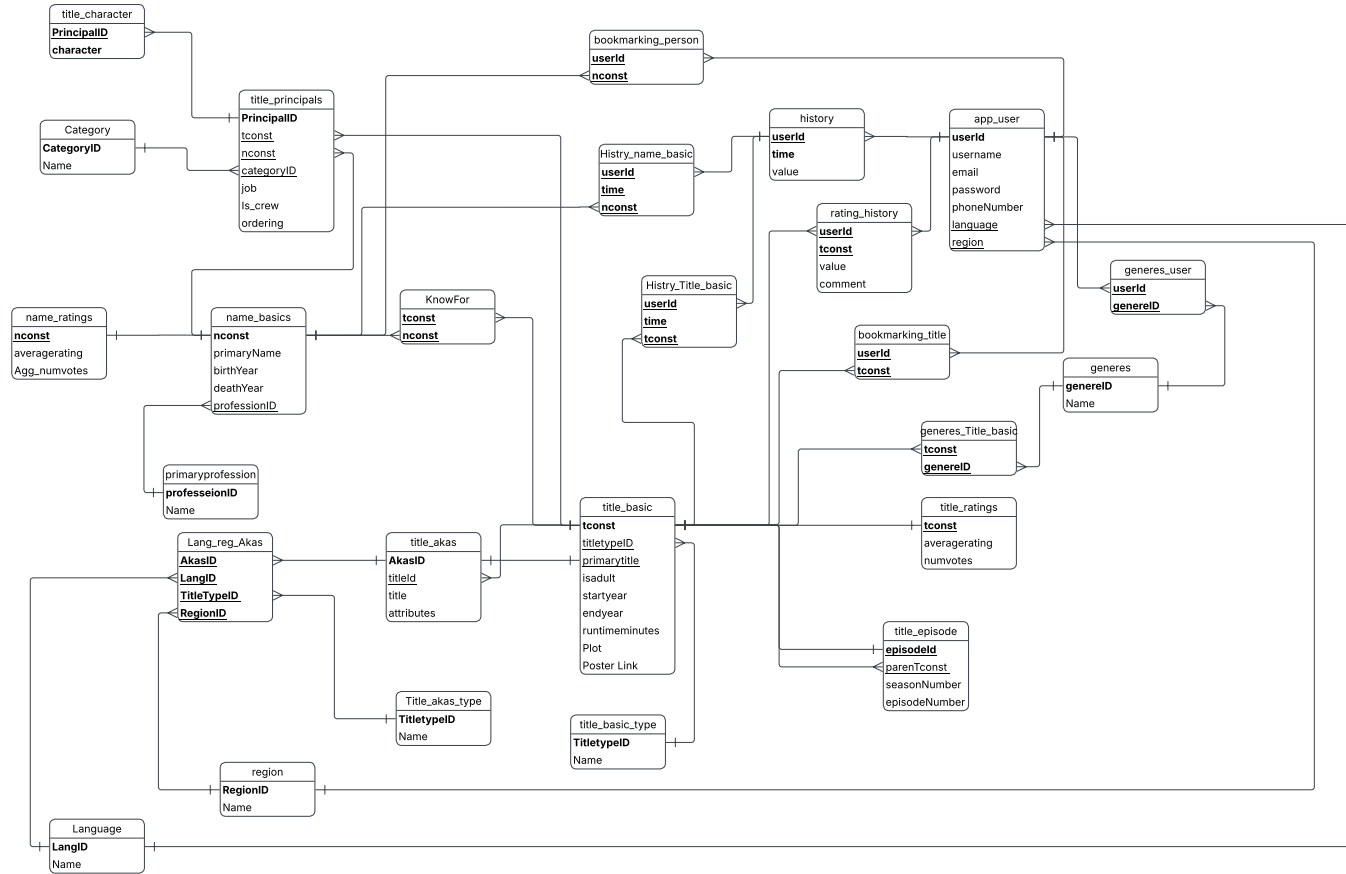


Figure 3: Reverse engineered diagram of the full database

6 Functionality

The functions are implemented in PostgreSQL (PL/pgSQL) and form the interface that the service layer uses. All functions that search or modify data take the user id as parameter, so that the framework is updated as a side effect of the call.

6.1 Framework functions

`create_user()` and `delete_user()` manage users; `bookmark_title()`, `bookmark_person()` and the corresponding `remove_` functions manage bookmarks; `get_bookmarked_titles()`, `get_bookmarked_persons()`, `get_search_history()` and `get_rating_history()` retrieve the user's data. Bookmarking uses `ON CONFLICT DO NOTHING`, so repeating a call is harmless instead of raising a key violation. The retrieval functions join in the title or person name, so the frontend does not need a second lookup. The search history is returned newest first. The password is stored as received; hashing is the responsibility of the calling layer.

6.2 Title search

`string_search()` returns `tconst` and `primarytitle` of the titles whose title or plot contains the search string. The insert into `history` is done inside the function, so a search and its log entry cannot be separated by a client.

`structured_string_search()` takes one string for each of title, plot, characters and person names and matches them case-insensitively as substrings against the corresponding columns. Characters and names are reached through `title_principals`, `title_character` and `name_basics` with `EXISTS` subqueries, which avoid duplicate result rows for titles with many matching people. An empty parameter is ignored, so every field is optional; a title is returned if it matches at least one non-empty parameter.

6.3 Rating

`rate()` takes a user, a title and a rating from 1 to 10. The average is stored together with the number of votes, so it can be updated incrementally. For a first rating by the user the new average is $(\bar{r}n + r)/(n + 1)$ and the number of votes increases by one. If the user has rated the title before, the old rating r_{old} is withdrawn and the new one inserted: $(\bar{r}n - r_{old} + r)/n$ with unchanged n . This is the second option suggested in the assignment; it lets users change their mind, and it is consistent because `rating_history` has the key `(userid, tconst)`. The range 1–10 is enforced by the `CHECK` constraint of `rating_history`. Since the function runs as one transaction, the update of `title_ratings` is rolled back if the insert is rejected.

6.4 Names, co-players and name ratings

`search_names()` finds people by case-insensitive substring of the name and logs the search. `co_players()` joins `title_principals` with itself on the title, excludes the person, and counts the shared titles per co-player in decreasing order.

The name rating of Section 3 is computed when the database is built. To keep it correct when titles are rated, the trigger `update_name_rating_trigger` fires after each update of `title_ratings` and recomputes the weighted average and total votes of all people involved in

that title. `popular_actors_in_movie()` uses the stored rating to list the people of a title in decreasing order of popularity.

6.5 Similar movies

Two titles are considered similar if they share genres, and the similarity is the number of shared genres. `similarity_search()` returns the titles with the highest similarity to a given title, limited by a parameter. Alternatives are the overlap of cast and directors, closeness in rating or year, or a combination. Shared genres were chosen because they are easy to explain and only need the small junction table `genres_title_basic`. The measure favours titles with many genres and gives many ties; normalising by the number of genres (Jaccard similarity) would remove the first effect.

6.6 Text retrieval over the inverted index

The table `wi` contains one row for each occurrence of a word in a title (title, plot, characters and person names). The following functions use it.

- **Frequent person words.** `search_by_exact_name()` finds the titles of the person, collects their words from `wi`, and returns the most frequent words with their counts.
- **Exact match.** `exact_match_search()` returns the titles that contain all keywords: group by title and require that the number of distinct matching words equals the number of keywords (relational division).
- **Best match.** `any_match_search()` ranks titles by the number of distinct keywords they contain, in decreasing order.
- **Word-to-words.** `words_query()` finds the titles that contain any keyword and returns the most frequent words in those titles, which serves as a weighted keyword list for the query.
- **Weighted.** The table `weights` is a weighted inverted index with TF-IDF weights $w_{t,k} = \text{tf}_{t,k} \cdot \ln(N/\text{df}_k)$, where $\text{tf}_{t,k}$ is the number of occurrences of word k in title t , df_k the number of titles containing k and N the number of titles. A word that occurs in few titles thus counts more than a common one. `weights_search()` sums the weights of the query words for each title and orders by the sum.

`weights` is a table and not a view because computing the weights requires aggregating the whole of `wi`, which is too slow to repeat for every query. The price is that the table must be rebuilt if `wi` changes.

7 Indexing

PostgreSQL creates an index for each primary key and unique constraint, but not for foreign key columns. The primary keys of the junction tables are composite, for example (`tconst`, `genreid`) in `genres_title_basic`, and such an index only supports lookups that start with its first column. The index script therefore adds B-tree indexes for the access paths used by the functions of Section 6; all of them serve equality predicates and joins.

- `title_principals(nconst)` and `title_principals(tconst)`: this is the central relationship table between titles and people. `co_players()` joins it with itself on the title and starts from a person; `popular_actors_in_movie()`, the name search in `structured_string_search()` and the trigger of Section 6.4 look up all people of a title, and the trigger also aggregates over all titles of a person.
- `name_basics(primaryname)`: `co_players()` and `search_by_exact_name()` find a person by exact name.
- `genres_title_basic(genereid)`: `similarity_search()` joins two rows on the genre, and the primary key only supports lookups by title.
- `knowfor(nconst)`: the primary key starts with `tconst`, but a person page needs the known-for titles of a person.
- `title_akas(titleid)`: the alternative titles of a title, needed on every title page.
- `title_episode(parentconst)`: the episodes of a series; the primary key `episodeid` only covers the opposite direction.

An index costs storage and slows down writes. Here the cost is small: the indexed tables belong to the movie data, which is loaded once and afterwards only read, while the writes of the application go to the framework tables, which are accessed through their primary keys.

The indexes do not help the substring searches of `string_search()`, `structured_string_search()` and `search_names()`, because a pattern with a leading wildcard cannot use a B-tree; a trigram index (extension `pg_trgm`) would be needed. The text retrieval functions filter `wi` and `weights` on the column `word`, so indexes on these columns are the next candidates for improvement.

8 Testing

A single test script calls every function of Section 6 with example arguments. For the functions that modify data, it selects the affected rows before and after the call. The output of the script is saved in a file; both files are delivered together with the report. The tests only show that the code runs and gives plausible results.

9 Evaluation

- **Rounding.** `averagerating` is stored with one decimal and is updated from the rounded value, so the error can accumulate over many calls to `rate()`. Recomputing from `rating_history` is not an option, since that table contains only the local ratings and not the original IMDb votes.
- **Derived data.** `name_ratings` and `weights` are derived data. The first is kept consistent by the trigger as long as titles are rated through `rate()`; the second must be rebuilt if `wi` changes. This is the price of fast queries.